Software Testing: Concept Rubric

1. Unit Testing

Unit testing checks the smallest testable part of the software, such as a single function, method or class, in isolation from the rest of the system. It is written and run by the developers while they code. Other parts that the unit depends on are replaced with stubs or mocks so that only the unit itself is tested. Unit tests are fast and automated, using tools such as JUnit or pytest. They find logic errors early, when they are cheapest to fix.

2. Integration Testing

Integration testing checks that separate units or modules work correctly when they are combined. It focuses on the interfaces and the data passed between components, and it finds interface mismatches and wrong assumptions between modules. It is done after unit testing. Common approaches are big-bang, top-down and bottom-up integration.

3. System Testing, including Performance and Security Testing

System testing checks the complete, fully integrated system against the specified requirements. It covers both functional requirements and non-functional requirements. Performance testing measures speed and stability, for example response time, load testing under expected use and stress testing under heavy use. Security testing looks for vulnerabilities, such as weak authentication, broken access control, injection attacks and unprotected data. System testing is run in an environment that is similar to production, usually by an independent test team, after integration testing is finished.

4. Smoke Testing versus Sanity Testing

Smoke testing is a quick, shallow set of tests run on a new build to confirm that the most critical functions work and that the build is stable enough for further testing. If the smoke test fails, the build is rejected. Sanity testing is a narrow, focused check run after a small change or a bug fix, to confirm that the specific change works and that nothing obvious has broken around it. In short, smoke testing is wide and shallow and is done on every new build, while sanity testing is narrow and deeper and is done after a small change.

5. Black-Box Testing versus White-Box Testing

Black-box testing checks the behaviour of the software from the outside, using inputs and expected outputs, without knowing the internal code. Techniques include equivalence partitioning and boundary value analysis. White-box testing checks the internal structure of the code, so the tester needs to know the code, and it aims for coverage such as statement coverage and branch coverage. In short, black-box asks whether the software does what it should, and white-box asks whether the code paths have been exercised.

6. Regression Testing

Regression testing means re-running previously passed tests after a change, such as a bug fix or a new feature, to confirm that existing functionality has not been broken by the change. It protects

the software against new defects introduced by changes. Regression tests are usually automated and are run again for every release or build.